Python Control Flow: for, while and range

The for loop

A for loop iterates over the items of any iterable such as a list, string, tuple, dictionary or range object. It is the preferred way to process every item in a collection.

```
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)

for ch in "hello":
    print(ch)
```

range() and off-by-one errors

range(stop) produces numbers from 0 up to but not including stop. range(start, stop) begins at start. range(start, stop, step) moves by step. The stop value is always excluded, which is the main source of off-by-one errors.

```
list(range(5))          # [0, 1, 2, 3, 4]
list(range(1, 6))       # [1, 2, 3, 4, 5]
list(range(10, 0, -2))  # [10, 8, 6, 4, 2]

# Sum 1 to 10 inclusive: stop must be 11
total = 0
for i in range(1, 11):
    total += i
```

When indexing a list by position, use range(len(seq)), never range(len(seq) + 1). Better still, avoid indexes: use for item in seq, or for i, item in enumerate(seq) when you need both the position and the value.

The while loop

A while loop repeats as long as its condition is true. The body must change something that eventually makes the condition false, otherwise the loop runs forever.

```
count = 0
while count < 5:
    print(count)
    count += 1       # forgetting this line causes an infinite loop
```

break, continue and else

break exits the nearest loop immediately. continue skips the rest of the current iteration. A loop else block runs only if the loop finished without hitting break, which is useful for search loops.

```
for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            break
    else:
        print(n, "is prime")
```

Modifying a list while looping over it

Removing or inserting items in a list while iterating over the same list skips elements and gives confusing results. Iterate over a copy, or build a new list with a list comprehension.

```
nums = [1, 2, 3, 4, 5, 6]
for n in nums:
    if n % 2 == 0:
        nums.remove(n)        # BUG: skips items

nums = [n for n in nums if n % 2 != 0]   # fix
```

Useful helpers

enumerate(seq) yields (index, item) pairs. zip(a, b) walks two sequences together. reversed(seq) iterates backward. sorted(seq) returns a new sorted list. Using these makes loops shorter and avoids index mistakes.